

Московский авиационный институт (национальный исследовательский университет)
Институт № 8 «Компьютерные науки и прикладная математика»
Кафедра № 806 «Вычислительная математика и программирование»

Разработка и исследование пулов потоков с механизмом `work-stealing` на языке C++

Выпускная квалификационная работа студента СпецВО-магистратуры

Студент группы М8О-214СВ-24: Спиридонов Кирилл Анатольевич
Научный руководитель: д.ф.-м.н., доцент Морозов Александр Юрьевич

Москва — 2026



- В начале 2000-х годов рост тактовой частоты процессоров достиг физических пределов — индустрия перешла к многоядерным архитектурам
- Флагманские серверные процессоры содержат **сотни ядер**; параллелизм стал повседневной инженерной реальностью
- Классический пул потоков с **единой глобальной очередью** плохо масштабируется: очередь превращается в узкое место при большом числе потоков
- Стандартная библиотека **C++17** не **предоставляет** готовой реализации пула потоков

Проблема

Отсутствует реализация пула потоков с механизмом work-stealing, построенная исключительно на стандарте C++17 без внешних зависимостей и подкреплённая систематическим экспериментальным исследованием



Цель — разработать библиотеку на языке C++, реализующую пул потоков с механизмом work-stealing, и экспериментально исследовать её производительность в сравнении с классическим пулом потоков

Задачи:

- Проанализировать существующие подходы и реализации пулов потоков
- Спроектировать архитектуру библиотеки с единым интерфейсом для двух реализаций
- Реализовать SimpleThreadPool и WorkStealingPool без внешних зависимостей
- Верифицировать корректность посредством автоматических тестов и Thread Sanitizer
- Провести сравнительное исследование производительности и сформулировать практические рекомендации



Дано:

- Многоядерная вычислительная система
- Поток задач с неизвестной заранее структурой нагрузки
- Теоретическая оценка Блюмофе–Лейзерсона:

$$T_P \leq \frac{T_1}{P} + O(T_\infty)$$

- Алгоритм Chase–Lev для двусторонней очереди

Требования к решению:

- Язык — C++17, без внешних зависимостей
- Единый интерфейс `submit()` для обеих реализаций
- Корректность при конкурентном доступе (отсутствие гонок данных)

Необходимо:

Библиотека с двумя взаимозаменяемыми реализациями пула потоков и экспериментальное сравнение их характеристик



Модель Блюмофе–Лейзерсона:

- Параллельное вычисление — ориентированный ациклический граф
- T_1 — работа (однопроцессорное время)
- T_∞ — глубина (критический путь)
- Гарантия жадного work-stealing:

$$T_P \leq \frac{T_1}{P} + O(T_\infty)$$

Закон Амдала:

$$S_P \leq \frac{1}{\alpha + \frac{1-\alpha}{P}}$$

Принцип работы deque:

Владелец (LIFO, нижний конец)

`push(task)` `pop(task)`

В типичном случае — без конкуренции

Вор (FIFO, верхний конец)

`steal(task)`

Забирает «старые» крупные задачи

Асимметрия обеспечивает низкую конкуренцию и хорошую локальность данных



Три независимых модуля:

WorkStealingDeque

Двусторонняя очередь.
Асимметричный доступ
владельца и воров.
Синхронизация: `std::mutex`

SimpleThreadPool

Единая глобальная очередь.
`std::mutex` +
`std::condition_variable`

WorkStealingPool

Единый интерфейс:

```
template<typename F, typename...  
Args> auto submit(F&& f,  
Args&&... args) -> std::future<  
invoke_result_t<F,Args...>;
```

Управление жизненным циклом:

- Конструктор: создаёт рабочие потоки
(по умолчанию —
`hardware_concurrency()`)
- Деструктор: graceful shutdown —
дожидается завершения всех задач
- Классы не копируемы и



Рабочий цикл потока с индексом i :

1 Своя очередь:

`queues[i].pop(task)`

Если есть — выполнить, перейти к 1

2 Кража: до 32 попыток

случайно выбрать жертву $j \neq i$,

`queues[j].steal(task)`

Если удалось — выполнить, перейти к 1

3 Ожидание: `cv.wait_for(1мс)`

Предикат: есть задачи или флаг остановки

4 Завершение: если флаг

Нетривиальные проблемы реализации:

Гонка в деструкторе SimpleThreadPool

`notify_all()` до входа потока в `wait()` $\Rightarrow$ уведомление теряется.

Решение: устанавливать флаг под мьютексом

Переполнение `size_t` в `pop()`

`bottom_ - 1` при `bottom_ == 0` $\Rightarrow$ `segfault`.

Решение: проверять `bottom == top` до вычитания



28 автоматических тестов (Google Test):

Набор тестов	Тестов
WorkStealingDeque	12
SimpleThreadPool	6
WorkStealingPool	10
Итого	28

Ключевые тесты:

- SingleElementRaceCondition — 10 000 итераций: владелец и вор одновременно претендуют на последний элемент; задача выполняется ровно один раз
- DestructorWaitsPending — деструктор дожидается всех задач
- HeavyLoadAllTasks — 100 000 задач под нагрузкой

Результат



Тестовая машина:

Параметр	Значение
Логических ядер	16
Тактовая частота	5102,71 МГц
Кэш L1	32 КиБ × 8
Кэш L2	1024 КиБ × 8
Кэш L3	16384 КиБ × 1
ОС	Linux
Компилятор	Clang, C++17
Оптимизация	-O2

Два сценария нагрузки:

Сценарий 1: много мелких задач

10 000 задач, каждая — атомарный инкремент (≈ 5 нс).

Измеряет накладные расходы планировщика.

Сценарий 2: неравномерная нагрузка

10 000 задач: половина — атомарный инкремент,

половина — `sleep_for(200 мкс)`.

Целевой сценарий для work-stealing.



Сценарий 1: много мелких задач

Потоков	Simple, мс	WS, мс
1	4,52	4,39
2	3,76	4,86
4	6,77	7,93
8	14,2	14,8
16	12,5	15,2

Обе реализации **деградируют** с ростом числа потоков: накладные расходы планировщика многократно превышают полезную работу

Сценарий 2: неравномерная нагрузка

Потоков	Simple, мс	WS, мс
1	1268	1266
2	638	635
4	317	317
8	159	237
16	80,3	98,1

При 16 потоках SimpleThreadPool быстрее по реальному времени, но WorkStealingPool потребляет **в 3,8 раза меньше CPU** (2,75 мс против 10,4 мс)



Почему WorkStealingPool медленнее по реальному времени?

- 1 WorkStealingDeque использует `std::mutex` — операции владельца **не** lock-free.
Теоретическое преимущество локальных очередей не достигается
- 2 Механизм ожидания:
`cv.wait_for(1мс)` — потоки засыпают при отсутствии задач; пробуждение с задержкой до 1 мс
- 3 SimpleThreadPool пробуждает потоки **мгновенно** через `condition_variable`

Практические рекомендации:

SimpleThreadPool

Предпочтителен при жёстких требованиях к **латентности**: высоконагруженные серверы, интерактивные приложения

WorkStealingPool

Предпочтителен при требованиях к **энергоэффективности**: фоновые вычисления, мобильные устройства, системы с ограниченным энергопотреблением.



- 1 Спроектирована и реализована** модульная библиотека на чистом C++17 без внешних зависимостей: SimpleThreadPool, WorkStealingPool, WorkStealingDeque с единым интерфейсом submit()
- 2 Верифицирована корректность:** 28 тестов пройдены, Thread Sanitizer не обнаружил гонок данных; выявлены и устранены нетривиальные дефекты
- 3 Установлен фундаментальный компромисс:** SimpleThreadPool — меньшая латентность (80,3 мс при 16 потоках), WorkStealingPool — в 3,8 раза меньше CPU (2,75 мс против 10,4 мс) при неравномерной нагрузке
- 4 Научная новизна:** систематически исследовано влияние механизма ожидания на соотношение реального и процессорного времени; выявлены условия, при которых блокирующая синхронизация в deque нивелирует преимущества work-stealing

Перспективы

Lock-free deque (алгоритм Chase–Lev + hazard pointers); NUMA-топологическая



Полный исходный код размещён в открытом репозитории GitHub.

